



Kaunas University of Technology
Department of Applied Informatics

Machine Learning

Laboratory work 1

Student: Sevastianova Polina

Lecturers: Prof. dr. Dalia Čalnerytė

Kaunas, 2026

Task

1. Dataset(Philippines Housing Market)
2. Data Quality Analysis
3. Linear Regression Model

Data description and quality analysis

Housing_v2.csv dataset is given. It is a real estate dataset containing numerical and categorical features aimed at predicting price of property. There are 9 columns: Description, Location, Price, Bedrooms, Bathrooms, Floor Area, Land Area, Latitude, Longitude. Missing values appear in dataset (834) and it is 6.95% of whole dataset. There is no wrong values such as Price or Floor Area. For duplicates there were 114 elements and all of them were removed.

There are 7 numerical columns and 2 categorical ones.

Numerical	Categorical
Price	Description
Bedrooms	Location
Bathrooms	
Floor Area	
Land Area	
Latitude	
Longitude	

Column Description has free and unstructured text, almost 1300 rows are unique and it causes high cardinality. So, it was removed at the beginning of dataset.

For numerical values:

	Price	Bedrooms	Bathrooms	FloorArea	LandArea	Latitude	Longitude
Total	1500	1500	1500	1500	1500	1500	1500
Missing values	49	63	80	37	23	289	289
Missing %	3,2	4,2	5,3	2,4	1,5	19,2	19,2
Cardinality	831	13	18	411	413	459	459
Minimum	300000	1	1	22	27	6.12	120.22
Maximum	2500000000	13	43	2200	25000	17,39	125,80
Q1	4902000	3	2	67,56	81	14,37	120,90
Q3	25000000	4	4	250	240	14,91	121,07
Mean	32822180	3,62	3	213,93	274,17	14,24	121,20
Median	9200000	3	3	142	121	14,53	121,03
Std	101201200	1,37	1,85	236,52	889,24	1,75	0,99

Dataset contains 1500 property records with mix of details and coordinates used to predict real prices. Like most data, it is a bit messy and requires some cleaning. For location(latitude/longitude) are missing 289 rows which makes up 19,27% of dataset. Other columns have minor gaps.

Price data is heavily skewed. While median price sits at 9200000, mean has gone up to 32822180 due to extreme outlier of 2.5 billion. Both Floor Area and Land Area have huge ranges. For example, land area has median of just 121 but peaks at 25000 for massive properties. But I cleaned these up with IQR capping as well.

Mean: Add all variables together and divided be total count. It gives overall balance point of data.

$$\text{Mean} = \frac{1}{n} \sum_{i=1}^n x_i$$

Median: Sort all values and pick exact middle one. If there is even number of rows, it is average of two middle numbers.

Q1: 25th percentile. Once data is sorted, it is median of lower half of dataset.

Q3: 75th percentile. It is median of upper half of dataset, meaning 75% of values are below this point.

Standard deviation: Measures how much individual values spread out. Low std means houses/rooms are similar, while high means they vary widely.

$$s = \sqrt{\frac{\sum_{i=1}^n (x_i - \text{Mean})^2}{n - 1}}$$

For categorical values:

	Location
Total	1500
Missing values	4
Missing %	0,26
Cardinality	664
Mode1	Emilio Vergara Hwy Santa Arcadia, Cabanatuan
Mode1 Frequency	71
Mode1 %	4,75
Mode2	MacArthur Hwy Anonas, Urdaneta
Mode2 Frequency	59
Mode2 %	3,95%
Std	101201200

1500 total entries recorded for property locations. Only 4 missing values which are easily handled.

Cardinality is 664 that means that there are 664 completely unique location names in dataset. Because that is extremely high for categorical column, using One-Hot Encoding would create hundreds of sparse columns and break model. That's why target encoding was used instead.

Mode is simply most frequent occurring value in the column – the location that appears the highest number of times.

Outliers were calculated with Interquartile Range(IQR) method. Outliers will not be dropped or deleted, they will be handled using values at the calculated lower and upper bounds. In real life datasets contain luxury properties, so if I simply delete them I will lost valuable information for high-end price prediction and my method will not ruin linear regression with their huge impact.

Price: 162 outliers identified, driven by luxury listing and extreme pricing.

Bedrooms: 125 outliers, representing non-standart properties.

Bathrooms: 12 outliers, showing extreme bathrooms counts.

Floor Area: 113 outliers, capturing large houses.

Land Area: 156 outliers, showing massive land.

Latitude: 190 outliers, making properties on geographic edges of target region.

Longitude: 327 outliers, properties spread across longitude lines.

Project uses Multi Linear Regression to predict property prices based on multiple structural inputs. Model assumes that target price(y) has linear relationship with our feature variables(xi):

$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$, where β_0 is intercept, β_i represents weights of each feature, and ϵ is error term.

Optimization methods:

Parameters were calculated using both Ordinary Least Squares via matrix pseudo-inverse and iterative Gradient Descend algorithm that updates weights by stepping down error gradient.

Adding a column of ones (bias term) is necessary to allow model to calculate intercept β_0 automatically using standard matrix multiplication.

1. Ordinary Least Squared

It is analytical approach to solving linear regression. Instead of looping iteratively, it calculates the exact global minimum of cost function in single mathematical step.

For solving β intercepts standard normal equation:

$$\beta = (X^T X)^{-1} X^T y$$

Features can be collinear or redundant, meaning matrix $X^T X$ might not be fully invertible. To prevent errors, I used pseudo-inverse(np.linalg.pinv), which computes stable generalized inverse even when matrix is singular.

2. Gradient descend

It is iterative numerical optimization algorithm. It starts with initial random weights and updates them by moving in opposite direction of gradient until it reaches the lowest error point. For calculating gradient:

$$\text{grad} = \frac{1}{m} X^T (X\theta - y)$$

In each epoch(iteration), every parameter is updated simultaneously using learning rate that controls step size:

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

Model training strategy

To build normal model without data leakage or overfitting, our pipeline follows strict steps:

- 1) Train-Test split: dataset is split with 80/20 ratio to test model on completely unseen data.
- 2) Missing value imputation: all missing values were filled with median or mode. Median was used because it is resistant to extreme outliers and skewed data because it represents exact middle value when data is sorted. Mode was used for selecting the most frequently occurring value in that category.
- 3) Outlier capping: extreme values in numerical features are capped using $(Q_1 - 1.5 \times IQR)$ and $(Q_3 + 1.5 \times IQR)$ to prevent skewed data from distorting optimization.
- 4) Feature engineering: I created meaningful new variables like Total_Rooms, Total_Area, Area_Per_Room and Floor_To_Land_Ratio to give model better predictive signals.
- 5) Encoding & Normalization: high cardinality locations are handled via target encoding. Redundant columns like Longitude are dropped to minimize multicollinearity. Remaining categories use One-Hot encoding, and numeric features are min-max normalized.

	Parameter	Least squared	Gradient desc	Absolute difference
Intercept,	b0	-2.661012e+06	-2.661012e+06	1.764856e-07
	b_1	6.215446e+04	6.215446e+04	1.939188e-07
	b_2	9.351290e+05	9.351290e+05	5.180482e-08
	b_3	7.215416e+06	7.215416e+06	4.749745e-08
	b_4	1.758555e+06	1.758555e+06	1.937151e-07
	b_5	2.007876e+06	2.007876e+06	2.095476e-08
	b_6	5.859392e+05	5.859392e+05	2.316665e-08
	b_7	4.787729e+06	4.787729e+06	8.754432e-08
	b_8	-4.076221e+05	-4.076221e+05	2.797460e-07
	b_9	-4.558121e+04	-4.558121e+04	2.912493e-07

This table compares the model coefficients found by two different methods: analytical and iterative. The parameters for both approaches are virtually identical and agree up to the sixth decimal place, and the absolute difference is little values of order of 10^{-7} . This result proves that the gradient descent algorithm I wrote works absolutely correctly, successfully reached the global minimum of the loss function and produces values identical to the exact mathematical formula of OLS.

Training and Testing results

- 1) **Cross-Validation:** using 5-fold-cross-validation on training set, I compared 3 feature setups: all variables, significant variables($p < 0.05$) and top 3 variables.

Coefficient Statistical Significance Table

	Feature	Coefficient	Std. Error	t-statistic	p-value
	Intercept	-2.661012e+06	1.072157e+06	-2.481924	0.013216
	Bedrooms	6.215446e+04	1.007644e+06	0.061683	0.950827
	Bathrooms	9.351290e+05	1.367863e+06	0.683642	0.494346
	Floor Area	7.215416e+06	1.478390e+06	4.880592	0.000001
	Land Area	1.758555e+06	1.916813e+06	0.917437	0.359116
	Latitude	2.007876e+06	1.077377e+06	1.863671	0.062635
	Total_Rooms	5.859392e+05	7.320021e+05	0.800461	0.423617
	Total_Area	4.787729e+06	9.942979e+05	4.815185	0.000002
	Area_Per_Room	-4.076221e+05	3.083518e+06	-0.132194	0.894855
	Floor_To_Land_Ratio	-4.558121e+04	2.278066e+06	-0.020009	0.984040
	Location_encoded	4.265918e+07	9.370235e+05	45.526261	0.000000

Out of all features, only three crossed significance threshold: Floor Area, Total_Area and Location_encoded. Their extremely high t-statistics and near-zero p-values prove they have non-random impact on property prices. Location_encoded has by far highest t-statistic, demonstrating that neighbor positioning is single most dominant factor driving real estate values in dataset. Features like individual Bedrooms, Bathrooms, and engineered ratios yielded high p-values, meaning they carry random noise rather than distinct predictive signal when combined.

For cross-validation such results were obtained:

	Model Case	Mean MSE	Std Dev MSE
	ii. Top 3 Variables	2.615397e+13	1.548476e+13
i. Significant Variables		2.615397e+13	1.548476e+13
	iii. All Variables	2.663313e+13	1.607379e+13

Model with 3 top and significant variables get same results because they consist of same variables. Short model gives better results, has less average error (2.615×10^{13}) comparing to whole model. It means that less number of variables provide higher accuracy on validation blocks.

Standard deviation for short model works better. This indicates that the model “jumps” less between different parts of the data and that it works more stably on new subsets.

2) **Test set evaluation:** assessing unseen data demonstrate another tendency.

	Model	MSE	RMSE	MAE
	ii. Top 3 Variables	1.404466e+14	1.185102e+07	9.603466e+06
i. Significant Variables		1.404466e+14	1.185102e+07	9.603466e+06
	iii. All Variables	1.379758e+14	1.174631e+07	9.503820e+06

Top 3 and significant variables show same results. Simultaneously whole model achieves better results on tests. While cross-validation proved that feature reduction effectively stabilized the model and removed noise during the training stages, keeping all variables in the full model allowed us to capture additional microsignals and provide slightly higher generalization accuracy on this particular test split.

3) **Custom vs Library comparison:**

Model & Features	Implementation	MSE	RMSE	MAE
Top 3 / Significant	Custom (Gradient Descent)	1.404466e+14	1.185102e+07	9.603466e+06
Top 3 / Significant	Scikit-Learn	1.404520e+14	1.185125e+07	9.603776e+06
All Variables	Custom (Gradient Descent)	1.379758e+14	1.174631e+07	9.503820e+06
All Variables	Scikit-Learn	1.379754e+14	1.174629e+07	9.503324e+06

Custom gradient descend implementation achieved an MSE of 1.379758×10^{14} , matching Scikit-Learn’s library implementation almost down to exact digit.

4) **Single prediction example:**

Running pipeline on a sample

```
example_record = {
    "Bedrooms": 3,
    "Bathrooms": 2,
    "Floor Area": 120,
    "Land Area": 200,
    "Location": "Quezon City",
    "Latitude": 14.65,
}
```

Market valuation of \$29,313,850.33, confirming that model handles real-world inference correctly.

Conclusion

Proper handling of missing values, target encoding for high-cardinality locations and IQR outlier capping stabilized dataset without data leakage. While full model captured minor extra variance on test set, cross-validation proved that reducing features to significant variables removed unnecessary model complexity, prevents overfitting. The high parity between manual linear regression/gradient descend loops and scikit-learn validates core mathematical integrity of entire machine learning pipeline.